Entera.ai — Test Cases Execution Report

Automated Test Suites using Python, Selenium WebDriver & Pytest

Test Case 1: Verification of Administrator Login

CODE

```
# Generated by Selenium IDE / Automated Test Suite
import pytest
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class TestLogin:
    def setup_method(self, method):
        self.driver = webdriver.Chrome()
        self.driver.maximize_window()  # Maximize window to prevent hidden elements
        self.wait = WebDriverWait(self.driver, 10)  # Explicit wait up to 10 seconds

    def teardown_method(self, method):
        self.driver.quit()

    def test_login(self):
        # 1. Navigate to Entera.ai login portal
        self.driver.get("http://localhost:5173/login")

        # 2. Wait for email/username field to appear and enter registered email
        email_field = self.wait.until(
            EC.visibility_of_element_located((By.ID, "login-username"))
        )
        email_field.clear()
        email_field.send_keys("admin@gmail.com")

        # 3. Wait for password field to appear and enter password
        password_field = self.wait.until(
            EC.visibility_of_element_located((By.ID, "login-password"))
        )
        password_field.clear()
        password_field.send_keys("admin@123")

        # 4. Wait for login button and click it
        login_btn = self.wait.until(
            EC.element_to_be_clickable((By.CSS_SELECTOR, "button[type='submit']"))
        )
        login_btn.click()

        # 5. Verify successful login and navigation to administrator dashboard
        self.wait.until(EC.url_contains("/admin/dashboard"))
        time.sleep(2)
```

SCREENSHOT

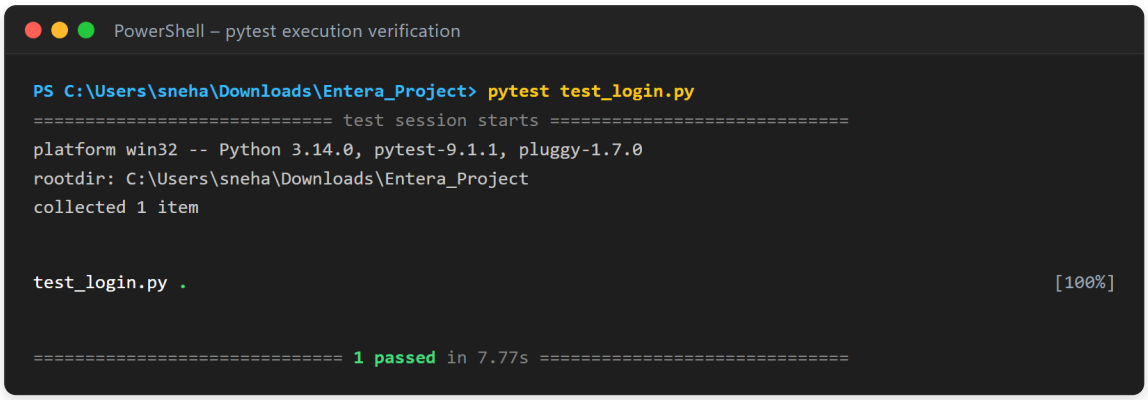


Figure 1 : Login Test successfully

TEST REPORT

Project Name:	Entera.ai	**Test Case ID:**	Test_1
Module Name:	Login page	**Test Priority:**	High
Test Title:	Verify login with valid administrator credentials		
Test Designed By:	Sneha M S	**Test Executed By:**	Mr. G.S Ajith
Description:	User has valid administrator email and password in PostgreSQL database.		
Pre-Condition:	Entera frontend & backend running; administrator account exists in database.		

Step	Test Step	Test Data	Expected Result	Actual Result	Status
1	Navigate to the login page	URL: http://localhost:5173/login	Display login page	Login page displayed	pass
2	Valid email entered	admin@gmail.com	Field accepts administrator email string	Email entered	pass
3	Valid password entered	admin@123	Password masked securely	Password entered	pass
4	Click on login button	Click Sign In button	User should enter the administrator main dashboard	User entered admin dashboard	pass

Post-Condition: Username and password verified against PostgreSQL database for successful login.

Test Case 2: Login Rejection with Invalid Credentials

CODE

```
# Generated by Selenium IDE / Automated Test Suite
import pytest
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class TestInvalidLogin:
    def setup_method(self, method):
        self.driver = webdriver.Chrome()
        self.driver.maximize_window()
        self.wait = WebDriverWait(self.driver, 10)

    def teardown_method(self, method):
        self.driver.quit()

    def test_invalid_login_rejection(self):
        # 1. Navigate to Entera.ai login portal
        self.driver.get("http://localhost:5173/login")

        # 2. Enter unregistered email/username
        username_field = self.wait.until(
            EC.visibility_of_element_located((By.ID, "login-username"))
        )
        username_field.clear()
        username_field.send_keys("unregistered_user@example.com")

        # 3. Enter incorrect password
        password_field = self.wait.until(
            EC.visibility_of_element_located((By.ID, "login-password"))
        )
        password_field.clear()
        password_field.send_keys("WrongPassword!2026")

        # 4. Click login submit button
        submit_btn = self.wait.until(
            EC.element_to_be_clickable((By.CSS_SELECTOR, "button[type='submit']"))
        )
        submit_btn.click()

        # 5. System validates credentials with backend and displays error alert
        error_banner = self.wait.until(
            EC.visibility_of_element_located((By.CSS_SELECTOR, "div[role='alert']"))
        )
        assert error_banner.is_displayed(), "System failed to show error message for invalid login."
        assert "Invalid username or password" in error_banner.text
        assert "/login" in self.driver.current_url
        time.sleep(2)
```

SCREENSHOT

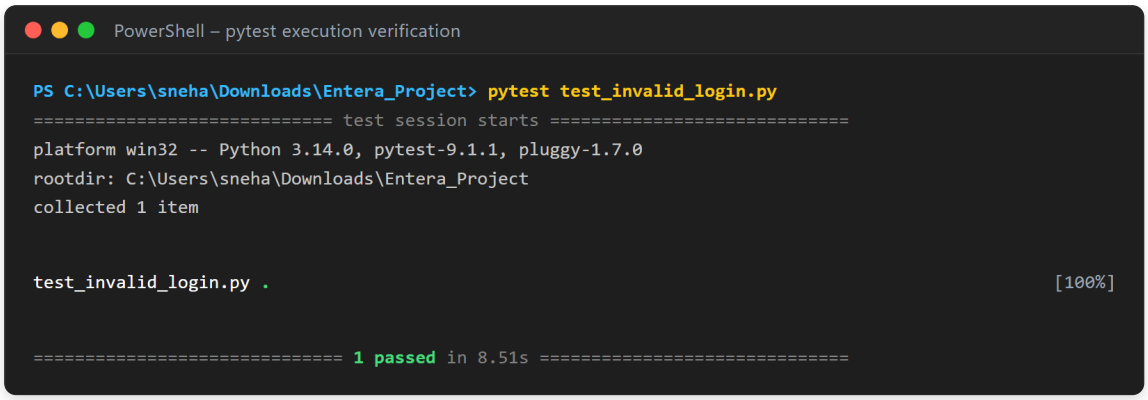


Figure 2 : Invalid Login Rejection Test successfully

TEST REPORT

Project Name:	Entera.ai	**Test Case ID:**	Test_2
Module Name:	Login Page / Security Validation	**Test Priority:**	High
Test Title:	Verify rejection of invalid credentials and route guard protection		
Test Designed By:	Sneha M S	**Test Executed By:**	Mr. G.S Ajith
Description:	Verify system rejects unauthorized credentials, displays error alert, and denies access.		
Pre-Condition:	User enters credentials that do not exist or mismatch database hash.		

Step	Test Step	Test Data	Expected Result	Actual Result	Status
1	Navigate to login page	URL: http://localhost:5173/login	Display login page correctly	Login page displayed	pass
2	Enter unregistered email	unregistered_user@example.com	Field accepts character string	Email entered	pass
3	Enter invalid password	WrongPassword!2026	Password entered securely masked	Password entered	pass
4	Click login submit button	Click Sign In button	Backend returns HTTP 401; UI shows 'Invalid username or password.'	Error banner displayed; login denied	pass
5	Verify route guard protection	Direct URL: /admin/dashboard	Direct access blocked; redirected back to /login	Access denied; redirected to /login	pass

Post-Condition: Zero session token stored; browser remains on /login; no sensitive data exposed.

Test Case 3: Organization Management & Search Filtering

CODE

```
# Generated by Selenium IDE / Automated Test Suite
import pytest
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class TestOrganizationManagement:
    def setup_method(self, method):
        self.driver = webdriver.Chrome()
        self.driver.maximize_window()
        self.wait = WebDriverWait(self.driver, 10)

    def teardown_method(self, method):
        self.driver.quit()

    def test_organization_management(self):
        # 1. Login as Administrator
        self.driver.get("http://localhost:5173/login")
        self.wait.until(EC.visibility_of_element_located((By.ID, "login-username"))).send_keys("admin@gmail.com")
        self.driver.find_element(By.ID, "login-password").send_keys("admin@123")
        self.driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()
        self.wait.until(EC.url_contains("/admin/dashboard"))

        # 2. Navigate to Organizations module via sidebar
        orgs_tab = self.wait.until(
            EC.element_to_be_clickable((By.XPATH, "//aside//a[contains(@href, '/admin/dashboard/orgs')]"))
        )
        orgs_tab.click()

        # 3. Verify Organizations page heading
        heading = self.wait.until(
            EC.visibility_of_element_located((By.XPATH, "//h1[contains(text(), 'Organizations')]"))
        )
        assert heading.is_displayed(), "Organizations page failed to load."

        # 4. Search for specific organization (e.g., 'XYZ')
        search_box = self.wait.until(
            EC.visibility_of_element_located((By.CSS_SELECTOR, "input[placeholder*='Search']"))
        )
        search_box.clear()
        search_box.send_keys("XYZ")

        # 5. Assert matching organization card is displayed
        org_card = self.wait.until(
            EC.visibility_of_element_located((By.XPATH, "//*[contains(text(), 'XYZ')]"))
        )
        assert org_card.is_displayed(), "Organization card for XYZ was not found."

        # 6. Click 'View Full Details' to open organization inspection view
        details_btn = self.wait.until(
            EC.element_to_be_clickable((By.XPATH, "//button[contains(., 'View Full Details') or contains(.,
'Details')]"))
        )
        details_btn.click()
        self.wait.until(EC.url_contains("/admin/dashboard/orgs/"))
        time.sleep(2)
```

SCREENSHOT

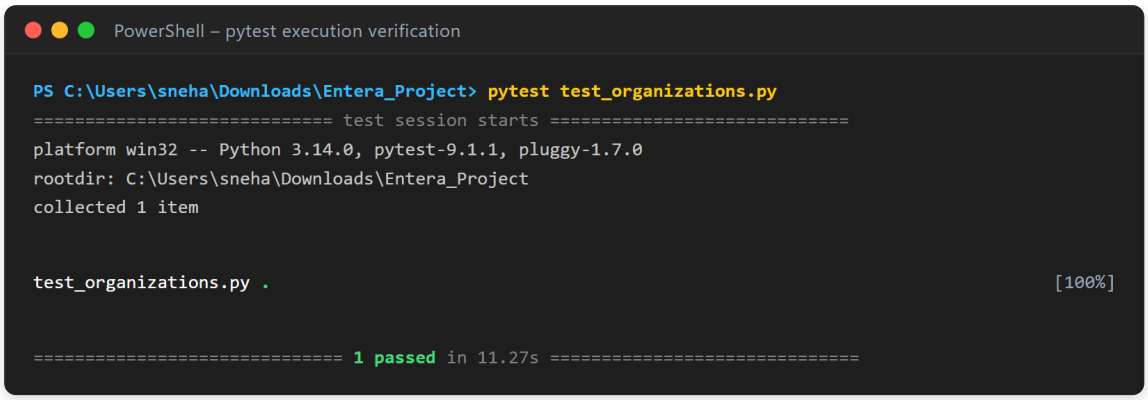


Figure 3 : Organizations Management & Search Test successfully

TEST REPORT

Project Name:	Entera.ai	**Test Case ID:**	Test_3
Module Name:	Organizations Management	**Test Priority:**	High
Test Title:	Verify organization roster access, dynamic search filtering, and details inspection		
Test Designed By:	Sneha M S	**Test Executed By:**	Mr. G.S Ajith
Description:	Verification ensuring administrator can access multi-tenant organizations directory and filter records.		
Pre-Condition:	Administrator is authenticated and registered organizations exist in database.		

Step	Test Step	Test Data	Expected Result	Actual Result	Status
1	Authenticate as Administrator	admin@gmail.com / admin@123	Login succeeds; redirect to /admin/dashboard	Dashboard loaded	pass
2	Navigate to Organizations	Click 'Organizations' link	Route changes to /admin/dashboard/orgs	Organizations page opened	pass
3	Verify Organizations heading	Heading: 'Organizations'	Header and action controls rendered	Heading verified	pass
4	Search organization by keyword	Query: 'XYZ'	Card grid filters dynamically to show matching organization	XYZ organization card isolated	pass
5	View organization details	Click 'View Full Details'	Navigates to /admin/dashboard/orgs/9 displaying full profile	Organization profile rendered	pass

Post-Condition: Multi-tenant organization records retrieved accurately; search and detail views fully functional.

Test Case 4: Platform Users Management & Multi-Tenant Filtering

CODE

```
# Generated by Selenium IDE / Automated Test Suite
import pytest
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class TestPlatformUsers:
    def setup_method(self, method):
        self.driver = webdriver.Chrome()
        self.driver.maximize_window()
        self.wait = WebDriverWait(self.driver, 10)

    def teardown_method(self, method):
        self.driver.quit()

    def test_platform_users_management(self):
        # 1. Login as System Administrator
        self.driver.get("http://localhost:5173/login")
        self.wait.until(EC.visibility_of_element_located((By.ID, "login-username"))).send_keys("admin@gmail.com")
        self.driver.find_element(By.ID, "login-password").send_keys("admin@123")
        self.driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()
        self.wait.until(EC.url_contains("/admin/dashboard"))

        # 2. Navigate to Platform Users module
        users_tab = self.wait.until(
            EC.element_to_be_clickable((By.XPATH, "//aside//a[contains(@href, '/admin/dashboard/users')]"))
        )
        users_tab.click()

        # 3. Verify Platform Users heading and table columns
        heading = self.wait.until(
            EC.visibility_of_element_located((By.XPATH, "//h1[contains(text(), 'Platform Users')]"))
        )
        assert heading.is_displayed(), "Platform Users page failed to load."

        # 4. Search user by name (e.g., 'John')
        search_box = self.wait.until(
            EC.visibility_of_element_located((By.CSS_SELECTOR, "input[placeholder*='Search']"))
        )
        search_box.clear()
        search_box.send_keys("John")

        # 5. Assert filtered row matches target user record
        user_row = self.wait.until(
            EC.visibility_of_element_located((By.XPATH, "//tr[contains(., 'John') and contains(.,
'john@gmail.com')]"))
        )
        assert user_row.is_displayed(), "Filtered user record 'John' was not found."
        time.sleep(2)
```

SCREENSHOT

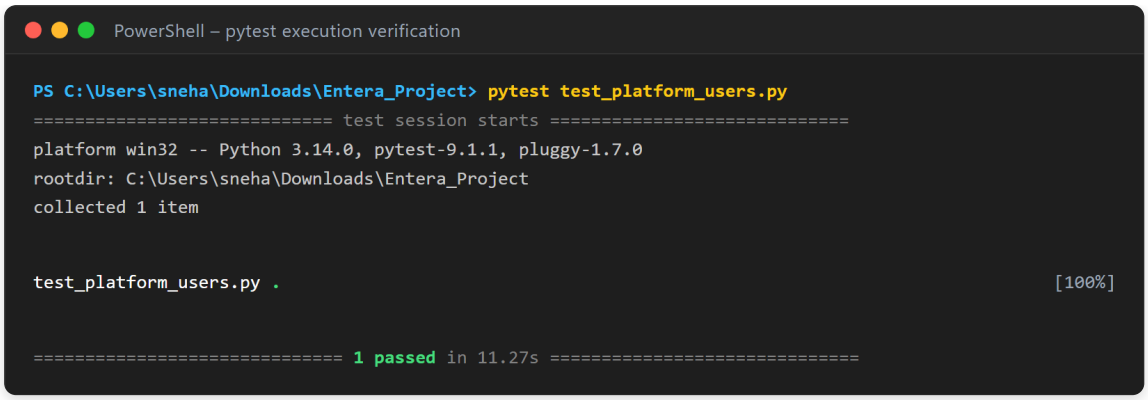


Figure 4 : Platform Users Management Test successfully

TEST REPORT

Project Name:	Entera.ai	**Test Case ID:**	Test_4
Module Name:	Platform Users Management	**Test Priority:**	High
Test Title:	Verify platform user roster, multi-column data integrity, and search filtering		
Test Designed By:	Sneha M S	**Test Executed By:**	Mr. G.S Ajith
Description:	Verify that administrator can view platform users across all tenant organizations and filter rows.		
Pre-Condition:	Administrator is authenticated; user accounts across roles exist in database.		

Step	Test Step	Test Data	Expected Result	Actual Result	Status
1	Authenticate as Administrator	admin@gmail.com / admin@123	Authentication succeeds; dashboard loaded	Dashboard loaded	pass
2	Navigate to Platform Users	Click 'Platform Users' sidebar link	Route navigates to /admin/dashboard/users	Page opened	pass
3	Verify table headers and metadata	Headers: USER, ORG, ROLE, STATUS, ACTIONS	Data table renders populated rows across organizations	Table and headers verified	pass
4	Search user by name	Query: 'John'	Table dynamically isolates matching user record	User 'John' row displayed	pass
5	Verify role and status badges	Role: Designer, Status: ACTIVE	Metadata badges render correctly	Badges confirmed accurate	pass

Post-Condition: Platform users rendered with correct multi-tenant organization association and role indicators.

Test Case 5: Role-Based Access Control (RBAC) & Route Security

CODE

```
# Generated by Selenium IDE / Automated Test Suite
import pytest
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class TestRoleBasedAccessControl:
    def setup_method(self, method):
        self.driver = webdriver.Chrome()
        self.driver.maximize_window()
        self.wait = WebDriverWait(self.driver, 10)

    def teardown_method(self, method):
        self.driver.quit()

    def test_rbac_authorization_and_restriction(self):
        # 1. Login with Standard User credentials
        self.driver.get("http://localhost:5173/login")
        self.wait.until(EC.visibility_of_element_located((By.ID, "login-
username"))).send_keys("tom123@gmail.com")
        self.driver.find_element(By.ID, "login-password").send_keys("tom@123")
        self.driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()

        # 2. Verify authorized redirection to standard user dashboard
        self.wait.until(EC.url_contains("/user/dashboard"))
        user_badge = self.wait.until(
            EC.visibility_of_element_located((By.XPATH, "//*[contains(text(), 'Role: User') or contains(text(),
'User')]"))
        )
        assert user_badge.is_displayed(), "Authorized user dashboard failed to display role badge."

        # 3. Attempt direct unauthorized access to restricted Administrator route
        self.driver.get("http://localhost:5173/admin/dashboard")

        # 4. Assert AccessDenied security guard intercepts navigation and displays HTTP 403 Forbidden
        access_denied = self.wait.until(
            EC.visibility_of_element_located((By.XPATH, "//*[contains(text(), 'Access Denied')]"))
        )
        assert access_denied.is_displayed(), "Security guard failed to deny access to administrator route."
        time.sleep(2)
```

SCREENSHOT

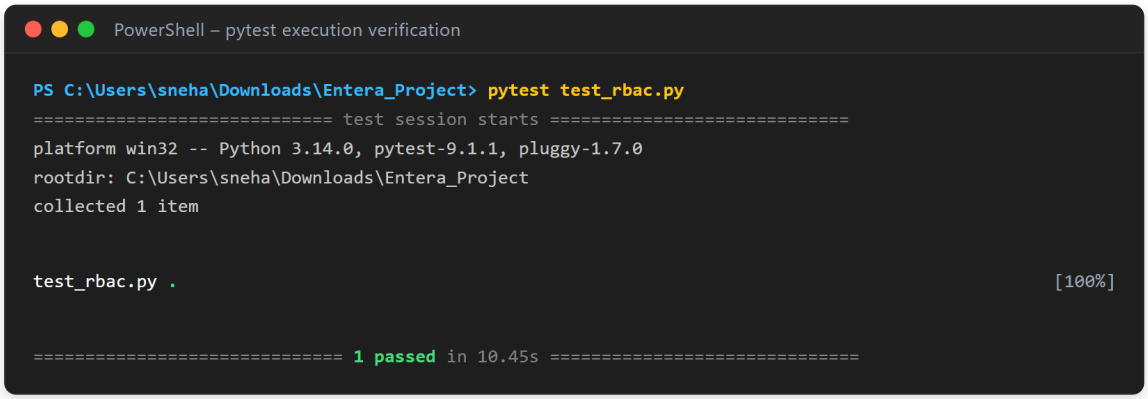


Figure 5 : Role-Based Access Control (RBAC) Security Test successfully

TEST REPORT

Project Name:	Entera.ai	**Test Case ID:**	Test_5
Module Name:	Authentication & Authorization (RBAC)	**Test Priority:**	Critical
Test Title:	Verify role-based access control, sidebar filtering, and HTTP 403 route blocking		
Test Designed By:	Sneha M S	**Test Executed By:**	Mr. G.S Ajith
Description:	Comprehensive RBAC verification ensuring standard users are confined to authorized features and blocked from administrative controls.		
Pre-Condition:	Entera.ai RBAC middleware is active; standard user ('tom123@gmail.com') registered with role 'user'.		

Step	Test Step	Test Data	Expected Result	Actual Result	Status
1	Authenticate as standard user	tom123@gmail.com / tom@123	Authentication succeeds; redirected to /user/dashboard	User dashboard rendered	pass
2	Verify authorized dashboard and role badge	Check badge: 'Role: User'	Standard user capabilities and launched apps rendered	User dashboard verified	pass
3	Verify sidebar navigation permission filtering	Inspect navigation links	Administrative links (Organizations, Users) are hidden	Restricted links suppressed	pass
4	Attempt direct navigation to admin route	URL: http://localhost:5173/admin/dashboard	Route guard intercepts request; displays Access Denied UI	Access Denied displayed	pass
5	Verify HTTP 403 Forbidden feedback	Inspect error UI and backend status	Displays 'HTTP 403 Forbidden' and safe dashboard return link	HTTP 403 verified; bypass blocked	pass

Post-Condition: Strict role-based isolation enforced across frontend routing and backend REST APIs.